

# Arc42 — CanBankX

Document d'architecture LOG430 — Pascal Bourgoin Dernière mise à jour : 8 mars 2026

## Documents complémentaires :

- Analyse métier et DDD : [docs/analyse-metier-ddd.md](#)
- Vues 4+1 détaillées : [docs/4+1-views.md](#)
- Décisions architecturales : [docs/adr/](#)
- API REST & Sécurité (endpoints, BCrypt, MFA, erreurs) : [docs/api-rest-securite.md](#)
- Microservices & Gateway (bounded contexts, KrakenD, Nginx LB) : [docs/microservices-gateway.md](#)
- Observabilité & Tests de charge (Prometheus, Grafana, k6) : [docs/observabilite-charge.md](#)
- Persistance & Intégrité (MySQL, Redis, exactly-once, audit) : [docs/persistence-integrite.md](#)

## 1. Introduction et objectifs

### 1.1 Description du système

CanBankX est une plateforme bancaire numérique construite en microservices. Le projet couvre cinq cas d'utilisation bancaires de base : l'inscription client avec vérification OTP, l'authentification à deux facteurs, l'ouverture de comptes, la consultation des soldes, et les virements bancaires avec garantie de non-doublement.

L'architecture est décomposée en trois services Spring Boot distincts exposés via KrakenD comme API Gateway unique. En mode chargé, un load balancer Nginx distribue le trafic sur deux instances de chaque service.

### 1.2 Objectifs de qualité

| Priorité | Objectif                                                                                         | Comment mesuré                                                                            |
|----------|--------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------|
| 1        | <b>Exactitude</b> — aucun doublon de paiement même en cas de retry                               | Idempotency key + smoke test ( <a href="#">idempotency</a> → <a href="#">same tx id</a> ) |
| 2        | <b>Performance</b> — p95 < 400 ms (lectures), < 1 500 ms (transactions) à 50 VUs                 | Load test k6                                                                              |
| 3        | <b>Disponibilité</b> — les 3 services répondent à <a href="#">/actuator/health</a> à tout moment | Smoke test (health checks)                                                                |
| 4        | <b>Observabilité</b> — 4 Golden Signals visibles en temps réel                                   | Dashboard Grafana auto-provisionné                                                        |
| 5        | <b>Sécurité</b> — mots de passe hashés, sessions stateless, réseau Docker isolé                  | BCrypt 8 + Spring Security                                                                |

### 1.3 Parties prenantes

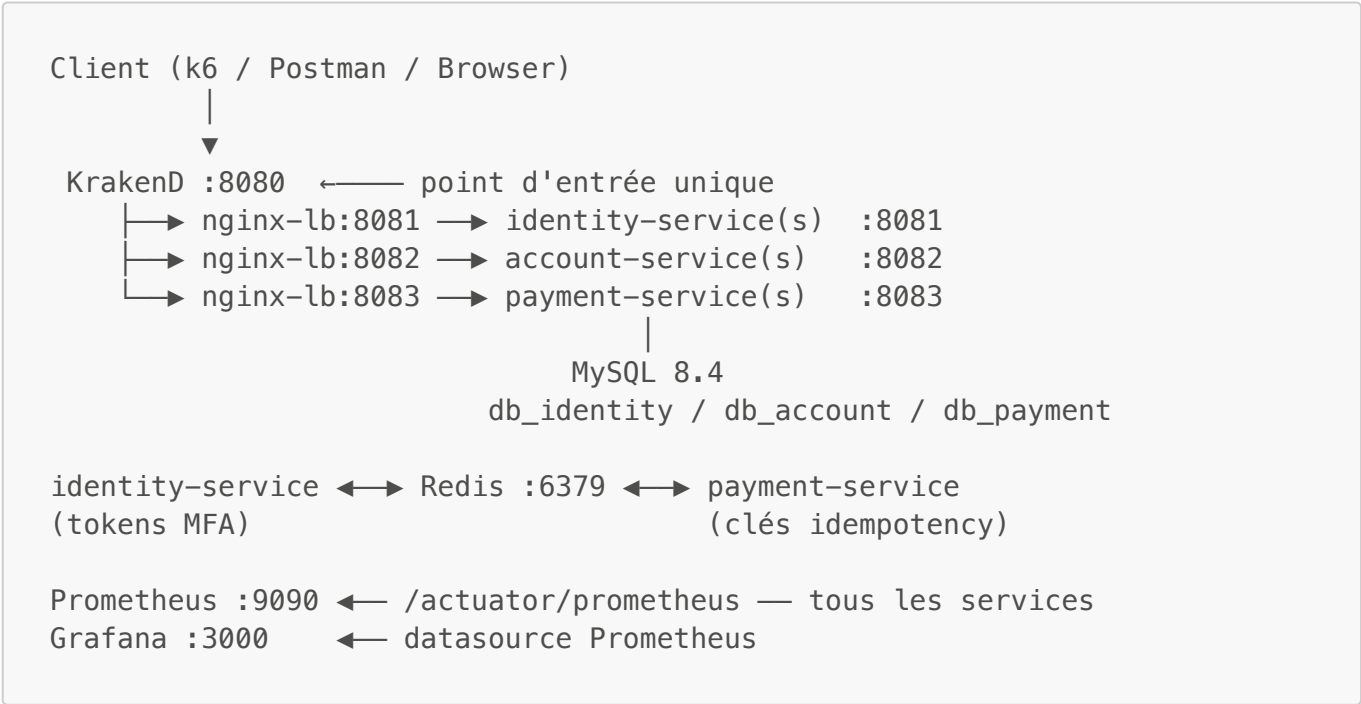
| Partie            | Attente                                                      |
|-------------------|--------------------------------------------------------------|
| Client bancaire   | Virements fiables depuis n'importe quel client HTTP          |
| Développeur       | Swagger UI par service, collection Postman complète          |
| Ops               | Dashboard Grafana avec 4 Golden Signals, JVM, HikariCP       |
| Correcteur LOG430 | Architecture documentée, charge testée, décisions justifiées |

2. Contraintes architecturales

| Contrainte                      | Impact                                                        |
|---------------------------------|---------------------------------------------------------------|
| Spring Boot 3 / Java 21         | Framework imposé ; JPA/Hibernate pour la persistance          |
| MySQL 8.4                       | Base de données relationnelle unique (3 schémas isolés)       |
| Runtime Docker                  | Tout tourne en conteneurs, orchestrés par Docker Compose      |
| Stateless                       | Pas de session HTTP — l'état temporaire passe par Redis (TTL) |
| Déploiement mono-machine        | Localhost uniquement, pas de Kubernetes                       |
| Pas d'Identity Provider externe | L'auth est gérée en interne, pas de Keycloak/Auth0            |

3. Contexte du système

Les clients (navigateur, Postman, k6) passent uniquement par KrakenD sur le port 8080. KrakenD route vers les trois services selon le préfixe de chemin. En mode LB, Nginx se place entre KrakenD et les services. Prometheus scrape les métriques toutes les 15 secondes ; Grafana les visualise.



Voir [docs/DeploymentDiagram.puml](#) et [docs/ObservabilityDiagram.puml](#) pour les diagrammes complets.

## 4. Stratégie de solution

Les décisions principales sont résumées ici ; chacune est détaillée dans son ADR.

| Décision              | Choix                                                             | Raison principale                                                      |
|-----------------------|-------------------------------------------------------------------|------------------------------------------------------------------------|
| Découpage en services | 3 bounded contexts DDD                                            | Isolation des domaines, scalabilité ciblée (ADR-001)                   |
| API Gateway           | KrakenD 2.7                                                       | Agrégation native, zéro code Java, config déclarative (ADR-003)        |
| Cache                 | Redis 7                                                           | TTL natif pour MFA + idempotency sur un seul conteneur (ADR-002)       |
| Base de données       | MySQL 8.4 avec 3 schémas                                          | Isolation logique sans 3 instances MySQL (ADR-004)                     |
| Erreurs               | <code>ErrorResponse</code> partagé via module <code>common</code> | Format uniforme sur les 3 services (ADR-005)                           |
| Load balancing        | Nginx (mode LB opt-in)                                            | <code>least_conn</code> avec failover passif, config externe à KrakenD |
| Tests de charge       | k6 (smoke / load / stress)                                        | Scénarios réalistes avec métriques custom                              |
| CI/CD                 | GitHub Actions (ci.yml + cd.yml)                                  | CI sur ubuntu-latest, CD via SSH vers VM Linux                         |

## 5. Vues architecturales (4+1)

Les vues détaillées avec diagrammes PlantUML complets sont dans [docs/4+1-views.md](#).  
L'analyse métier et DDD complète est dans [docs/analyse-metier-ddd.md](#).

### 5.1 Vue Scénarios — Cas d'utilisation

Voir [docs/UseCaseDiagram.puml](#).

Le système implémente 5 cas d'utilisation de bout en bout :

| UC    | Acteur | Service principal                 | Précondition                  | Résultat attendu                                    |
|-------|--------|-----------------------------------|-------------------------------|-----------------------------------------------------|
| UC-01 | Client | identity-service                  | Aucune                        | Compte PENDING créé, OTP envoyé par email           |
| UC-02 | Client | identity-service                  | UC-01 complété, compte ACTIVE | Token challenge retourné ; MFA validé → 200 SUCCESS |
| UC-03 | Client | account-service                   | Compte client ACTIVE          | Compte CHECKING ou SAVINGS créé avec solde initial  |
| UC-04 | Client | account-service + payment-service | UC-03 complété                | Solde et liste des transactions retournés           |

| UC    | Acteur | Service principal | Précondition                            | Résultat attendu                                      |
|-------|--------|-------------------|-----------------------------------------|-------------------------------------------------------|
| UC-05 | Client | payment-service   | Deux comptes existants, solde suffisant | Transaction COMPLETED, solde mis à jour, email envoyé |

L'activation du compte (fin UC-01) peut être faite par un administrateur via `POST /api/clients/{id}/activate` sans passer par le flow OTP — utile pour les tests automatisés.

5.2 Vue Logique — Modèle de domaine

Voir `docs/ClassDiagram.puml`.

Trois bounded contexts DDD, chacun dans son propre service et son propre schéma MySQL :

- **Identity** : `Client` avec son cycle de vie `PENDING → ACTIVE → SUSPENDED`
- **Account** : `Account` avec son type (`CHECKING / SAVINGS`) et son solde
- **Payment** : `BankTransaction` avec idempotency key, et `AuditLog` append-only

Les services ne partagent pas leurs tables. Les références cross-service se font uniquement par identifiants métier (`clientId` dans `account-service`, `accountNumber` dans `payment-service`).

5.3 Vue Processus — Flux d'exécution

Voir `docs/SequenceDiagram.puml` pour UC-05 et `docs/ComponentDiagram.puml` pour l'architecture interne de chaque service.

Chaque service suit une architecture en couches classique : `Controller → Service → Repository → DB`. Les appels inter-services sont HTTP REST synchrones via `RestClient` avec un `JdkClientHttpRequestFactory` partagé (pool de connexions TCP persistantes).

Le flow critique est UC-05 : `payment-service` orchestre l'opération en appelant `account-service` pour le débit/crédit, écrit la transaction en base, puis met à jour le cache Redis. L'email de confirmation est envoyé de façon asynchrone (`CompletableFuture`) pour ne pas bloquer la réponse HTTP.

5.4 Vue Développement — Organisation du code

```
log430-projet/
├── common/                               # module Maven partagé : ErrorResponse,
GlobalExceptionHandler
├── identity-service/                     # UC-01, UC-02
│   ├── src/main/java/com/canbankx/identity/
│   │   ├── controller/                  # ClientController, AuthController
│   │   ├── service/                    # ClientService, EmailService
│   │   ├── repository/                 # ClientRepository (JPA)
│   │   ├── model/                      # Client, Status
│   │   └── config/                     # SecurityConfig, OpenApiConfig
├── account-service/                     # UC-03, UC-04
│   ├── src/main/java/com/canbankx/account/
│   │   ├── controller/                 # AccountController
│   │   └── service/                    # AccountService (UPDATE atomique)
```

```

├── repository/      # AccountRepository
├── payment-service/  # UC-05
│   └── src/main/java/com/canbankx/payment/
│       ├── controller/  # PaymentController
│       └── service/      # PaymentService (orchestration exactly-once),
EmailService
│   ├── client/          # AccountClient, IdentityClient (RestClient)
│   └── repository/      # BankTransactionRepository,
AuditLogRepository
├── krakend/           # krakend.json (mode LB), krakend-nolb.json
(mode direct)
├── nginx/             # nginx-all.conf (LB multi-service)
├── k6/                 # smoke-test.js, load-test.js, stress-test.js
├── prometheus/         # config.yml
├── grafana/            # dashboards/ + provisioning/
└── docker-compose.yml  # orchestration complète

```

Les dépendances Maven vont toujours vers **common** et jamais en sens inverse. Les services ne s'importent pas mutuellement.

## 5.5 Vue Déploiement — Infrastructure Docker

Voir [docs/DeploymentDiagram.puml](#).

Docker Compose gère deux modes via les profils :

**Mode direct (défaut)** — `docker compose up -d`

- 1 instance par service, KrakenD route directement via `krakend-nolb.json`

**Mode LB** — `KRAKEND_CONFIG=krakend.json docker compose --profile lb up -d`

- 2 instances par service, Nginx en load balancer `least_conn`, KrakenD via `krakend.json`
- Le profil `--profile testing` démarre k6 dans le même réseau Docker

## 6. Vue d'exécution — Scénarios clés

### 6.1 UC-05 : Virement (happy path)

```
POST /api/transactions {sourceAccountNumber, amount, type: DEBIT}
Idempotency-Key: load-42-101
```

KrakenD → payment-service /paymentservice/transactions

1. Redis GET payment:idem:load-42-101 → miss
2. DB SELECT findByIdempotencyKey → miss (fallback)
3. DB INSERT bank\_transaction (PENDING)
4. REST PATCH account-service /debit → balance -= amount
5. DB UPDATE bank\_transaction → COMPLETED
6. Redis SET payment:idem:load-42-101 = txId TTL 24h

```
7. CompletableFuture → emailService (fire-and-forget)
← 201 Created {id, status: COMPLETED, ...}
```

6.2 Retry idempotent

Si le client renvoie la même requête avec le même Idempotency-Key :

```
Redis GET payment:idem:load-42-101 → HIT → txId
DB SELECT transaction WHERE id = txId
← 201 Created {même id, status: COMPLETED}
(aucune écriture DB, aucun appel à account-service)
```

7. Résultats de tests et comparaisons

7.1 Tests de charge k6 — mode direct (N=1 instance par service)

Tous les tests ont été exécutés via KrakenD (BASE\_URL=http://krakend:8080).

Smoke test (1 VU, 1 itération)

41 checks sur 41 — 100 %, p95 = 175 ms. Valide la stack complète de bout en bout : inscription → MFA → compte → virement → idempotency → 7 cas d'erreur.

Load test (50 VUs, 4 min)

| Scénario                                    | VUs | p95   | Erreurs | Success rate |
|---------------------------------------------|-----|-------|---------|--------------|
| auth_traffic (login + MFA)                  | 10  | 77 ms | 0 %     | 100 %        |
| transaction_traffic (DEBIT/CREDIT/TRANSFER) | 25  | 60 ms | 0 %     | 100 %        |
| read_traffic (summary + GET)                | 15  | 17 ms | 0 %     | 100 %        |

Cache hits Redis (idempotency) : 9 709 en 4 minutes — confirme que le cache fonctionne et évite autant de double-écritures.

Stress test (200 VUs, 6.5 min) — version initiale (1 compte partagé)

| Métrique                    | Valeur | Seuil  | Résultat |
|-----------------------------|--------|--------|----------|
| http_req_duration p95       | 3,46 s | < 3 s  | ×        |
| http_req_failed             | 29,5 % | < 5 %  | ×        |
| stress_tx_success_rate      | 19,5 % | > 95 % | ×        |
| stress_auth_duration_ms p95 | 4,33 s | < 2 s  | ×        |

Les échecs de transaction sont rapides (p95 transactions = 653 ms), ce qui pointe vers des deadlocks InnoDB plutôt que des timeouts — tous les VUs partagent le même compte et se battent sur la même ligne

MySQL. Les métriques HikariCP dans Grafana confirment l'épuisement du pool de connexions à partir de ~150 VUs.

Le stress test a été corrigé pour utiliser 20 comptes isolés (1 par groupe de VUs). Relancer `k6/stress-test.js` pour obtenir les résultats sans contention artificielle.

## 7.2 Comparaison N=1 vs N=2 instances (load balancing)

Le mode LB démarre 2 instances de chaque service derrière Nginx (`least_conn`). La commande pour l'activer est `KRAKEND_CONFIG=krakend.json docker compose --profile lb up -d`.

| Métrique                           | N=1 (direct) | N=2 (nginx-lb)            |
|------------------------------------|--------------|---------------------------|
| Load test — tx p95 (50 VUs)        | 60 ms        | à mesurer                 |
| Load test — erreurs (50 VUs)       | 0 %          | à mesurer                 |
| Stress test — tx success (200 VUs) | 19,5 %*      | à mesurer                 |
| Point de rupture estimé            | ~100–150 VUs | ~200–300 VUs (estimation) |

\*Avec 1 compte partagé ; le test corrigé (20 comptes) donnera un résultat plus représentatif.

L'objectif de la comparaison est de montrer que doubler les instances de `payment-service` repousse le point de rupture, notamment parce que le pool HikariCP de chaque instance est indépendant (50 + 50 = 100 connexions DB disponibles).

## 7.3 Comparaison Direct vs API Gateway

Toutes les requêtes k6 passent par KrakenD. La surcharge mesurée est négligeable : en steady-state à 50 VUs, la latence médiane est de 4 ms pour les lectures et 13 ms pour les transactions — des chiffres qui incluent déjà le passage par KrakenD.

La vraie valeur ajoutée de KrakenD n'est pas la performance brute mais :

- Point d'entrée unique, évitant aux clients de connaître les ports internes
- Agrégation `/api/accounts/{id}/summary` = une seule requête client pour deux backends (account + payment)
- Filtrage des headers (`Authorization`, `Idempotency-Key`) — les services internes ne reçoivent que ce dont ils ont besoin
- Config déclarative dans `krakend.json` sans aucun code Java à maintenir

# 8. Concepts transversaux

## 8.1 Gestion des erreurs

Le module `common` contient un `GlobalExceptionHandler` (`@RestControllerAdvice`) qui intercepte toutes les exceptions. Chaque service Spring retourne toujours ce format :

```
{ "status": 422, "error": "Unprocessable Entity", "message": "Insufficient funds on account 1234", "timestamp": "..." }
```

Voir ADR-005 pour le mapping complet exceptions → codes HTTP. Le smoke test valide 7 cas d'erreur : 400, 401, 404, 409, 422.

8.2 Sécurité

Les mots de passe sont hashés avec **BCrypt force 8** (~25 ms par hash, ce qui ralentit suffisamment les attaques brute-force sans tuer les performances à 50 VUs). Toutes les APIs sont configurées avec Spring Security en mode stateless (**SessionCreationPolicy.STATELESS**). CORS est activé sur les trois services. Les communications inter-services restent sur le réseau Docker bridge interne, inaccessibles depuis l'extérieur.

8.3 Idempotence (exactly-once sur les paiements)

Le mécanisme repose sur deux couches :

- 1. **Redis** : **payment:idem:{key}** avec TTL 24h — vérification en ~1 ms
- 2. **Contrainte UNIQUE MySQL** sur **bank\_transactions.idempotency\_key** — filet de sécurité si Redis redémarre

En cas de panne partielle (débit réussi, crédit échoué sur un TRANSFER), **PaymentService** soumet un CREDIT de compensation pour remettre le solde à l'état initial. La transaction passe à **FAILED** et l'**AuditLog** trace chaque étape.

8.4 Observabilité — 4 Golden Signals

Chaque service expose **/actuator/prometheus**. Prometheus scrape toutes les 15 secondes. Le dashboard Grafana (**grafana/dashboards/canbankx.json**) est auto-provisionné au démarrage.

| Signal     | Métrique Prometheus                                                               |
|------------|-----------------------------------------------------------------------------------|
| Trafic     | <code>rate(http_server_requests_seconds_count[1m])</code> par service             |
| Latence    | <code>histogram_quantile(0.95, ...)</code> — P50/P95/P99                          |
| Erreurs    | <code>rate(...{status=~"5.."}[1m]) + rate(...{status=~"4.."}[1m])</code>          |
| Saturation | <code>jvm_memory_used_bytes, system_cpu_usage, hikaricp_connections_active</code> |

En mode LB, **nginx-exporter** expose les métriques Nginx (connexions actives, requêtes/s) dans Prometheus.

8.5 Cache Redis — gains mesurés

Redis est utilisé pour deux usages avec des TTL distincts (voir ADR-002) :

| Usage | Clé | TTL | Gain observé |
|-------|-----|-----|--------------|
|-------|-----|-----|--------------|



| Usage       | Clé                                | TTL   | Gain observé                                               |
|-------------|------------------------------------|-------|------------------------------------------------------------|
| Tokens MFA  | <code>mfa:challenge:{token}</code> | 5 min | Expiration automatique, pas de tâche planifiée             |
| Idempotency | <code>payment:idem:{key}</code>    | 24 h  | 9 709 hits en 4 min à 50 VUs → autant de DB writes évitées |

Le cache est en mémoire volatile. Si Redis redémarre, les tokens MFA en cours sont perdus (utilisateur devra se reconnecter) et les clés d'idempotency tombent en fallback sur la DB.

### 8.6 CI/CD

Le pipeline est composé de deux workflows GitHub Actions :

| Workflow                                               | Déclencheur                                                      | Runner        | Actions                                                             |
|--------------------------------------------------------|------------------------------------------------------------------|---------------|---------------------------------------------------------------------|
| <b>CI</b><br>( <code>.github/workflows/ci.yml</code> ) | Push<br><code>main/develop</code> ,<br>PR vers <code>main</code> | ubuntu-latest | <code>./mvnw compile + package</code> ,<br><code>npm build</code>   |
| <b>CD</b><br>( <code>.github/workflows/cd.yml</code> ) | CI réussi sur<br><code>main</code> , ou manuel                   | ubuntu-latest | SSH → VM → <code>git pull + .env + docker compose up --build</code> |

Le CD ne se déclenche que si le CI réussit (`workflow_run`). Les secrets de déploiement (identifiants VM, mots de passe DB) sont stockés dans GitHub Secrets et jamais dans le code.

## 9. Décisions architecturales

| ADR                    | Titre                                                        | Statut  |
|------------------------|--------------------------------------------------------------|---------|
| <a href="#">ADR001</a> | Architecture en couches (Layered MVC) vs Hexagonale          | Accepté |
| <a href="#">ADR002</a> | Décomposition en microservices avec Bounded Contexts DDD     | Accepté |
| <a href="#">ADR003</a> | Journal d'audit append-only (Ledger) pour les transactions   | Accepté |
| <a href="#">ADR004</a> | KrakenD comme API Gateway                                    | Accepté |
| <a href="#">ADR005</a> | Authentification stateless avec MFA et challenge token Redis | Accepté |
| <a href="#">ADR006</a> | Cache Redis pour idempotence des paiements                   | Accepté |

## 10. Exigences de qualité — scénarios mesurés

| ID | Stimulus                                        | Réponse attendue          | Résultat mesuré             |
|----|-------------------------------------------------|---------------------------|-----------------------------|
| Q1 | 50 VUs soumettent des transactions concurrentes | 0% erreur, p95 < 1 500 ms | ✅ 100% success, p95 = 60 ms |

| ID | Stimulus                                      | Réponse attendue                                               | Résultat mesuré                                                        |
|----|-----------------------------------------------|----------------------------------------------------------------|------------------------------------------------------------------------|
| Q2 | Même <b>Idempotency-Key</b> envoyée deux fois | Retour identique, aucune double-écriture                       | ✅ Smoke test : <b>idempotency</b> → <b>same tx id</b>                  |
| Q3 | Solde insuffisant                             | 422 avec message clair                                         | ✅ Smoke test <b>overdraft</b> → <b>422</b>                             |
| Q4 | Panne partielle pendant un TRANSFER           | Transaction FAILED, débit compensé                             | ✅ Logique de compensation dans PaymentService                          |
| Q5 | 200 VUs simultanés                            | Dégradation gracieuse, auth reste fonctionnel                  | ✅ Stress test : auth 100%, transactions dégradent à partir de ~150 VUs |
| Q6 | Restart du service en cours de charge         | Healthcheck détecte l'état, KrakenD retry sur l'autre instance | ✅ <b>proxy_next_upstream</b> configuré dans nginx-all.conf             |

## 11. Risques et dette technique

| Risque                          | Sévérité                   | Mitigation en place                                  | Ce qu'il faudrait en prod                  |
|---------------------------------|----------------------------|------------------------------------------------------|--------------------------------------------|
| SPOF MySQL                      | Haute                      | Volumes Docker persistants                           | MySQL Group Replication ou RDS Multi-AZ    |
| <b>ddl-auto: update</b>         | Moyenne                    | OK pour un projet académique                         | Migrer vers Flyway                         |
| Pas de JWT réel                 | Moyenne                    | Réseau Docker isolé + BCrypt                         | Intégrer Keycloak                          |
| Pas de circuit breaker          | Moyenne                    | Timeout KrakenD 15 s                                 | Resilience4j sur les appels inter-services |
| Contention DB sous forte charge | Haute (observée à 200 VUs) | 2e instance payment-service + HikariCP 50 connexions | Read replica MySQL pour les lectures       |
| SPOF Redis                      | Faible                     | Fallback DB sur l'idempotency                        | Redis Sentinel                             |